

10384 Quantum Information - exercises, week 10

Pen-and-paper exercises

This series of exercises is meant to illustrate the workings of Simon's algorithm.

1. The numbers from 0 to 15 can be represented by 4 bits. Perform the bitwise XOR (or, bitwise addition modulo 2) with 6 for all $a \in \{0, \dots, 15\}$, i.e. calculate $a \oplus 6$ for all a up to 15. For example, $2 \oplus 6 = 0010 \oplus 0110 = 0100 = 4$.
Notice how this pairs up the numbers, which demonstrates the rule $(a \oplus b) \oplus b = a \oplus (b \oplus b) = a \oplus 0 = a$.
2. Calculate the inner product modulo 2, i.e. $a \cdot b = \sum_i a_i b_i \pmod{2}$, of the bitstring representing $b = 13$ with the bitstrings representing all numbers $a \in \{0, \dots, 15\}$.
3. Let four qubits be initialized in the state corresponding to the value 13, i.e. the state $|\psi\rangle = |1\rangle \otimes |1\rangle \otimes |0\rangle \otimes |1\rangle$. Now perform the Hadamard transform, $|\psi\rangle \rightarrow \hat{H}^{\otimes 4} |\psi\rangle$, directly on each of the four qubits and write the resulting state fully out as a superposition of 16 kets.
Does this match with the result of exercise 2 and the formula for Hadamard transforms of computational basis states?
4. Now write out the Hadamard transform of the state $|\phi\rangle = \frac{|11\rangle + |13\rangle}{\sqrt{2}}$.
Some of the 16 terms have now been cancelled. Can you see what the remaining terms have in common? What is the connection to the result of exercise 1?
5. Imagine that the result of exercise 4 were the output of an implementation of Simon's algorithm. There are now eight possible measurement outcomes. Pick a few of the possible measurement outcomes y and see if you can find the hidden bitstring s using the condition that $y \cdot s = 0 \pmod{2}$, with $\cdot$ being the bitwise inner product. There are different ways of finding this, but the simplest may be to pick one random outcome at a time and look at what values of s are compatible.
6. Create an oracle on 4+4 qubits that implements the two-to-one function defined by this table:

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$f(x)$	8	0	12	4	12	4	8	0	9	1	13	5	13	5	9	1

Computational exercises

7. Using QuTiP, set up Simon's algorithm together with your oracle. You only need to read out the first four qubits.

Find the probabilities for the 16 different measurement outcomes using `run_statistics`, confirming the results from above. If you feel adventurous, you could also try to simulate an actual measurement run of e.g. 1000 circuit samplings. For inspiration, look at [this page](#) of QuTiP's documentation.

8. **Computational complexity of classical algorithms:** Using the `%timeit` magic function in Jupyter, try to investigate the complexity of various functions of large integers, e.g. addition, multiplication, greatest common divisor, modular exponentiation ($x^k \pmod{N}$), primality test, prime factorisation, etc. You can e.g. use the functions `gcd`, `isprime` and `factorint` from [SymPy](#) by `from sympy.ntheory import isprime, factorint; from sympy import gcd`.

Here is an example that measures and plots how long it takes to multiply two random n -bit integers (i.e. between 2^{n-1} and 2^n) as a function of n from 1 to 1500:

```

from random import randint

nmax = 1500
timings = []

for n in range(1, nmax):
    x = randint(2**(n-1), 2**n)
    y = randint(2**(n-1), 2**n)
    t = %timeit -o -q -n 100 x*y
    timings.append(t)

fig, axs = plt.subplots(1, 2, figsize=(12,5))
axs[0].plot(range(1, nmax), [t.average for t in timings])
axs[1].loglog(range(1, nmax), [t.average for t in timings])
axs[1].loglog(range(200, nmax), 1.5e-12*np.arange(200, nmax)**2)

```

The `-o` parameter to `%timeit` tells it to output a result object, the `-q` parameter tells it to be quiet (not print the timing result), while the `-n 100` is the number of times to run the function to calculate an averaged timing.

When plotting, use `plot`, `semilogy` and `loglog` as appropriate, depending on the scaling behaviour.

Within the for-loop, you could also create another loop that picks new random numbers a few times and perform the timing on those, averaging the results. This could give a smoother time curve.